

Travelling Salesman Problem (TSP)

Ahmed Al-khlil & Anas Al-abdolkarem & Osama mohamed

June 12, 2026

1 السيناريو

تخيل أن هناك شركة توصيل لوجستية تمتلك مركبة توصيل واحدة، وانطلاقاً من المقر الرئيسي للشركة، يتعين على السائق زيارة عدد محدد من المدن (أو العملاء) لتسليم البضائع، ثم العوده في نهايه المطاف الى نقطه البدايه (المقر الرئيسي). ولتحقيق اعلى مستوى من الكفاءة وتقليل التكاليف الاضافيه، يواجه السائق عده شروط صارمه:

1. يجب زياره كل مدينه مره واحده فقط.
2. يجب تقليل اجمالي المسافه المقطوعه (او اجمالي زمن الرحله او الوقود) الى الحد الادنى.
3. يجب العوده الى نقطه الانطلاق (المقر الرئيسي).
4. يمنع بتاتا اغلاق حلقه جزئيه ببعض المدن ثم العوده الى نقطه البدايه والخروج مره اخرى لاكمال باقي المدن.

1.1 التعقيد الحسابي للمشكلة

تكمن الصعوبه البالغه في هذه المشكله عندما يبدأ عدد المدن بالارتفاع، حيث يزداد عدد المسارات الممكنه بشكل "عاملي" ($n!$). لنستعرض كيف تتضخم الاحتمالات:

- اذا كان على السائق زياره 3 مدن فقط، فان حساب المسار الاقصر سهل للغاية لان الاحتمالات محدوده.
- اذا ارتفع العدد الى 10 مدن، يقفز عدد المسارات المحتمل الى اكثر من 180000 مسار بديل.
- اما اذا كان هناك 20 مدينه فقط، فان عدد الاحتمالات ينفجر ليصل الى رقم خيالي من الكوينتيليونات (10^{18} مسار).

بسبب هذا النمو الأسّي الهائل في الاحتمالات، يصبح من المستحيل على الحواسيب العاديه حساب واختبار كل مسار على حده للوصول الى الحل المثالي مع زياده حجم البيانات. لذلك، تصنف "مشكلة البائع المتجول" (TSP) كواحد من أشهر المسائل غير الحتميه متعدد الحدود الصعبه جداً (NP-hard)، وهي المسائل التي يسهل يصاغتها وفهمها، ولكن يصعب جداً حلها بشكل مطلق عندما تصبح واسعه النطاق.

2.1 انواع المشكله وعدد الوصلات

تنقسم الى نوعين رئيسيين بتاءاً على اتجاه الطرق:

1. **مشكله البائع الجوال المتناظره Symmetric TSP**: هنا تكون الطرق ذات اتجاهين ومتساويه في التكلفه. اي ان المسافه من المدينه A الى المدينه B, تساوي المسافه من A الى B. اي $(c_{ij} = c_{ji})$. ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = \frac{n(n-1)}{2} ; \quad n \in N^*$$

2. **مشكله البائع الجوال غير المتناظره Asymmetric TSP**: وهنا تكون الطرق موجهة وتختلف تكلفها حسب الاتجاه $(c_{ij} \neq c_{ji})$. ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = n(n-1) ; \quad n \in N^*$$

3.1 طرق حساب وتمثيل البيانات

عند حساب المسافات بين المدن وتجهيزها قبل ادخالها الى الخوارزميه كي لا تستهلك قدره المعالج في تكرار حسابات غير مهمه, ولحساب المسافات بين اي نقطتين هناك عده قوانين رياضيه, منها قانون حساب المسافه الاقليديه وهي المسافه المباشره بين نقطتين, وتعطى بالعلاقه التاليه:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

ولتمثيل البيانات رياضيا او برمجياً يتم وضع البيانات في مصفوفه ثنائيه الابعاد 2D حجمها $n \times n$.

- السطر يعبر عن مدينه الانطلاق, والعمود يعبر عن مدينه الوصول.
- التقاطع بين السطر i والعمود يعبر عن قيمه التكلفه c_{ij} (المسافه او الزمن).
- القيم على القطر الرئيسي تكون دائماً صفراً او مالا نهائيه, لمنع السائق من الانتقال من المدينه الى المدينه نفسها.

4.1 اختيار الخوارزميه المناسبه

1.4.1 معايير المقارنه في اختيار الخوارزميه المناسبه

عند التعامل مع مشكله البائع المتجول TSP. لا يوجد مفهوم "الخوارزميه الافضل مطلقاً" دون النظر الى سياق البيانات وحجم المشكله. ان عمليه اختيار الخوارزميه المناسبه تحكمها

دائماً مقايضه اساسيه (Tread-off) بين ثلاثة عوامل رئيسيه وهي: الزمن المستغرق (Time), الموارد المتاحة (اي القدره الحسابيه والتخزينيه) (Calculation & Memory), والدقه المطلوبه (Accuracy).

بناءً على هذه الفلسفه, ولتحديد الخوارزميه التي تحقق التوازن الامثل بين كفاءه الاداء ودقه النتائج, قمنا باجراء مقارنة ممنهجه بين سته من ابرز الخوارزميات عالمياً, تتراوح بين الحلول الجشعه البسيطه, والبرمجه الديناميكيه المعقده, وصولاً الى الخوارزميات الهجينه المتطوره. وفقاً لخمسه معايير محوريه:

1. فئة الخوارزميه (Algorithm class): التصنيف البنيوي للخوارزميه.
2. فلسفه اتخاذ القرار (Descision logic): الاليه التي تتبعها الخوارزميه لاستكشاف المسارات.
3. التعقيد الحسابي والزماني (Time & Space complexity): مدى استهلاك الخوارزميه للوقت وموارد النظام مع نمو عدد المدن (n).
4. دقه الحل (Accuracy): مدى قرب مسار الناتج من الحل الاقصر المثالي.
5. مرونة البيانات (Data flexibility): القدره على التعامل مع القيود المختلفه والمسارات موجهة الاتجاه.

2.4.1 مقارنة الخوارزميات

• الجار لأقرب (Nearest Neighbor):

1. فئة الخوارزميه (Algorithm class): جشعه (Greedy Heuristic).
2. فلسفه اتخاذ القرار (Descision logic): اختيار المدينه الاقرب محلياً في كل خطوه دون النظر للمستقبل.
3. دقه الحل (Accuracy): سيئه الى متوسطه (غالباً اطول بحوالي 25% من الحل المثالي).
4. التعقيد الحسابي والزماني (Time & Space complexity): التعقيد الزماني اسي $O(n)$, والذاكره $O(n^2)$ أو $O(n)$.

• هليد-كارب (Held-Karp):

1. فئة الخوارزميه (Algorithm class): دقه فائقه (Exact/Dynamic Programming).
2. فلسفه اتخاذ القرار (Descision logic): تقسيم المشكله الى مشاكل فرعيه ابسط وتخزين حلولها لمنع تكرار العمليات الحسابيه وحساب الحل المثالي المطلق.
3. دقه الحل (Accuracy): الدقه مثاليه 100%, ولكن لاتستطيع معالجه اعداد مدن تزيد عن 32 مدينه تقريباً.
4. التعقيد الحسابي والزماني (Time & Space complexity): التعقيد الزماني انفجاري $O(2^n n)$, والتعقيد التخزيني $O(2^n n)$ هائل جداً.

• مستعمره النمل (Ant Colony Optimization):

1. فئة الخوارزميه (Algorithm class): ذكاء السراب (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): محاكاة سلوك النمل عبر افراز الفيرومون، والمسارات الاقصر تكتسب جذباً اقوى تدريجياً عبر ترسب الفيرومون.
3. دقه الحل (Accuracy): عاليه جداً (تقترب من المثاليه في الاحجام المتوسطه).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني $O(I \times A \times n^2)$ حيث I التكرارات و A عدد النمل. والتعقيد التخزيني $O(n^2)$.

• التطور الجيني (Generational Optimization):

1. فئة الخوارزميه (Algorithm class): جينيه تطوريه (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): توليد مجتمع من الحلول، ثم اجراء تزاوج (Crossover)، وطفرات (Mutation) للبقاء للاصلح والاقصر.
3. دقه الحل (Accuracy): متوسطه الى عاليه (تحتاج وقت طويل للتقارب).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني $O(I \times P \times n^2)$ حيث I التكرارات و P حجم المجتمع. والتعقيد التخزيني $O(nP)$.

• opt-2 & opt-3:

1. فئة الخوارزميه (Algorithm class): تحسين محلي (Local Search).
2. فلسفه اتخاذ القرار (Descision logic): قطع وتوصيل الوصلات بين المدن، حافتين في كل مره (2-opt)، او ثلاثه في كل مره (3-opt). وذلك لايجاد وصلات اقصر
3. دقه الحل (Accuracy): متوسطه الى جيده جداً
4. التعقيد الحسابي والزمني (Time & Space complexity): العقيد الزمني $O(n^2)$ for opt-2 و $O(n^3)$ for opt-3.

• LKH-3:

1. فئة الخوارزميه (Algorithm class): تحسين محلي متطور (Advanced Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): تقطيع وتوصيل للوصلات بشكل ديناميكي متغير تابع ل K من الوصلات، (opt-k) مع ميزات ذكيه لمنع السقوط في الحلول المحليه الفرعيه وتدعم القيود المعقده.
3. دقه الحل (Accuracy): قريه جداً من المثاليه (99% الى 100%).
4. التعقيد الحسابي والزمني (Time & Space complexity): في التعقيد الزمني، تقترب جداً من $O(n^{2.2})$. وفي التعقيد التخزيني $O(n)$ ذو كفاءه عاليه جدا في اداره الذاكره.

2 خوارزميه LKH-3

1.2 مدخل الى عبقرية خوارزميه LKH-3

في عام 1973، قدم العالمان Shen Lin و Brian Kernighan خوارزميه LK، والتي ابتكرت فكرة استخدام k-opt متغير. فبدلاً من تبديل عدد ثابت من الوصلات (مثل وصلتين

او ثلاث في كل مره), تغير الخوارزميه عدد تبديل الوصلات بناءً على المعطيات ديناميكياً للخروج من فخ الانماط دون المستوى الامثل (Suboptimal Patterns).

في عام 2000, احدث العالم Keld Helsgaun ثوره في هذا البحث عندما قدم خوارزميه LKH. وقد اضاف محرك رياضي قوي لتوجيه محرك البحث المحلي: The Held-Karp Lower Bound. وذلك بدمج Subgradient Optimaization مع محرك تبديل الوصلات LK swapping engine, بذلك اصبحت خوارزميه LKH اقوى خوارزميه حل لمشكله TSP على الاطلاق.

في عام 2017, اصدر العالم Helsgaun خوارزميه LKH-3, حيث طور عده اصدارات من المشكله, مثل اضافته خاصيه توجيه عده بائعين, نوافذ الوقت, والتكديس, مما جعلها تتراس على عرش الخوارزميات الاكفئ في حل المشكله عند ازدياد حجمها.

اسطاعه خوارزميه LKH-3 تحقيق سرعتها بتقسيم المشكله الى اجزاء اصغر مركزه وذات كفاءه عاليه.

2.2 الخطوة الاولى: تهيئه البيانات

في المرحله الاولى نعد بيانات المدن ونقوم بحساب الوصلات بينها ونخزنها في Arrays تسهل التعامل معها.

3.2 الخطوة الثانيه: The Held-Karp optimaization loop

1.3.2 القاعده الرياضيه: Lagrangian Relaxation

قبل تبديل اي وصله, تقوم LKH بتشغيل subgradient optimaization لحساب معاملات العقوبه او التحول للمسافه (penalized distances) لكل مدينه. وثم تنشئ بنيه بيانيه تدعى 1-Tree في كل دوره. اذا كانت مدينه معينه مزاره الكثير من المرات ($degree > 2$), قيمه الpenalty خاصيتها ستزداد, مما يجعل التوصيل معها مكلف اكثر, اما اذا كانت معزوله ($degree < 2$), تنقص قيمه الpenalty خاصيتها, بعد مئات من هذه الدورات, تؤدي هذه العمليه الى تسطح الرسم البياني, مما يرفع الحد الادنى حتى يصل الى ما دون الحل المثالي للمشكله مباشرةً.

يمكن تمثيل مشكله البائع المتجول برقم وحيد: وهو تقليل اجمالي تكلفه الرحله مع مراعاة القيد الذي ينص على ان كل مدينه يجب ان يكون لها وصلتين فقط ($degree = 2$) بالضبط دون انشاء حلقات فرعيه منفصله. ولكن بسبب اجبار الدرجات ان تكون 2 بالضبط لكل المدن في وقت واحد هو ما يجعل المشكله من تصنيف (NP-Hard). استخدم طريقه Held-Karp تكتيكاً يدعى (Lagrangian Relaxation). وبدلاً من اجبار قيد الدرجة بالقوه, نقوم بازاله القيود الثابته وننقلها الى داله تحسين الهدف (penalty). لاي مصوفه من الارقام الحقيقه $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, نعرف مصوفه من قيم π المعدله C' :

$$C'_{ij} = C_{ij} + \pi_i + \pi_j$$

حيث C_{ij} هي المسافة الجغرافية الحقيقية بين المدينة i والمدينة j . والقيم π_j و π_i هي مضاعفات لاغرانج. اذا جمعنا هذه القيم C'_{ij} لكل مدينة، تصبح التكلفة المعدلة لمجموعه مدن تحتوي N وصلات متغيره او تابعه بشكل منهجي تبعاً للدرجات الخاصه بكل مدينة d_{ij} :

$$\begin{aligned} \text{Penalized Cost} = W(\pi) &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} C_{i,j} + \sum_{i=1}^N d_i \pi_i \\ &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} (C_{i,j} + \pi_i + \pi_j) \end{aligned}$$

حيث E' هي مجموعه جزئية من مجموعه الوصلات E .

2.3.2 الية عمل الحلقة: Subgradient Ascent

تبدأ عملية التحسين باستخدام Subgradient لايجاد اقصى قيمه للحد الادنى، تعمل الحلقة على اربع خطوات متكرره:

1. حساب Minimum 1-Tree: باستخدام قيم الاوزان المعدله (اوزان العقوبه) C'_{ij} الحاليه، يتم تشغيل خوارزميه Kuskal عليها لايجاد شجره MST على $N - 1$ مدن، ثم نقوم بوصل المدينة المميزه (المعزوله) معها باستخدام اقصر وصلتين عندها (من حيث قيمه π_i ايضاً). بذلك يكون اجمالي الوزن المعدل الناتج معطى ب $W(\pi)$.

• السبب وراء اختيار البنيه 1-Tree: يعد إيجاد مسار صحيح لمسألة البائع المتجول TSP أمراً بالغ الصعوبة بسبب قيد صارم: يجب أن يكون حلقة واحدة متصلة حيث تتصل كل مدينة بمدينتين أخريين فقط. ويُعتبر إيجاد بنية ذات وزن أدنى وبهذه الخاصية تحدياً كابوساً من فئة NP-Hard.

تتجاوز شجرة 1-Tree هذه الصعوبة بتخفيف القواعد بما يكفي لتسهيل العمليات الحسابية، مما ينشئ بنية تعد بديلاً مثاليًا لمسار TSP. رياضياً، شجرة 1-Tree هي بنية بيانية مبنية على N مدينة تحتوي على N وصله بالضبط وحلقة مغلقة واحدة فقط، ويتم ذلك باستخدام حيلة تفكيك بسيطة من خطوتين:

(أ) أنشئ الشجرة الأساسية، واختر مدينة واحدة لتكون "المدينة الخاصة" (مثل المدينة 0). أزلها تمامًا من الخريطة، ثم أنشئ شجرة امتداد دنيا قياسية (MST) عبر المدن المتبقية $N-1$ باستخدام خوارزمية Kruskal.

(ب) بمجرد العثور على الشجرة الممتدة الدنيا، نعود إلى المدينة 0 ونقوم بتوصيلها قسرًا بالشجرة الممتدة الدنيا الناتجة باستخدام أرخص حافتين متاحيتين لها. بناءً على تكاليفك الحالية.

لأن الشجرة الممتدة الدنيا القياسية لا تحتوي على حلقات، وعقدة إضافية واحدة (المدينة 0) ستضمن إنشاء دورة واحدة بالضبط.

• لماذا تعد 1-Tree مرآة مثالية لمشكلة TSP؟
لنلق نظرة على المتطلبات الهيكلية لمسار صالح في مسألة البائع المتجول TSP مقارنة بـ 1-Tree:

- تحتوي جولة TSP الصحيحة على N من الحواف، متصلة بالكامل، ولها دورة واحدة فقط، ولكل عقدة درجة تساوي 2 بالضبط.

- تحتوي الشجرة أحادية البعد على N حافة متصلة بالكامل، ولها دورة واحدة فقط، لكن درجات عقدها يمكن أن تكون أي قيمة (درجة 1 أو أكثر)، وهو عدد الحواف المتصلة بها.

وهذا يعني أن مسار TSP الصالح هو ببساطة نوع خاص من بنية 1-Tree، حيث تكون درجة كل مدينة مساوية لـ 2، كما أن قواعده وبنيته البسيطة تجعل عملية بنائه سهلة للغاية.

• عملية استخراج الشجرة الممتدة الدنيا MST:
لاستخراج الشجرة الممتدة الدنيا (Minimum Spanning Tree (MST)، نستخدم خوارزمية Kruskal، بما أننا نبني شجرة أحادية، فإننا نتجاهل مدينة واحدة، مما يترك لنا عدد $N - 1$ مدينة، وفي نظرية الرسوم البيانية (Graph Theory)، تتطلب الشجرة الممتدة الدنيا التي تربط مجموعة من الرؤوس V دائمًا $V - 1$ حافة بالضبط لربطها جميعًا دون تكوين حلقة مغلقة، وبالتالي سيكون إجمالي عدد الحواف $(N - 1) - 1$ (المدينة المميزة التي عُزلت).

• الاداء الاساسيه: The Disjoint Set Union (DSU):
أثناء قيام خوارزمية Kruskal بفحص الوصلات المرتبة (من الأرخص إلى الأعلى)، تطرح سؤالاً بسيطاً: "إذا أضفت هذه الوصلة، فهل سننشئ حلقة مغلقة غير قانونية؟"، يجيب هيكل المجموعة المنفصلة (Disjoint Set) على هذا السؤال فوراً من خلال تتبع المدن المتصلة بالفعل بفروع هيكلية. ولذلك يستخدم مصفوفتين بسيطتين:

- **parents**: يتتبع الجذر المباشر (الأصل) لكل مدينة في مجموعتها، إذا كانت المدينة هي أصلها، فهي جذر مجموعتها المستقلة.

- **ranks**: يتتبع عمق فروع الشجرة لمجموعة ما لضمان أنه عند دمج مجموعتين، يتم توصيل المجموعة الأقصر أسفل المجموعة الأطول.

عند بدء تشغيل الخوارزمية، تكون كل مدينة كياناً منفرداً ومعزولاً تماماً؛ إذ لا تعرف أي مدينةٍ منها المدن الأخرى. وتشكل كل مدينة "دائرة اجتماعية" مستقلة خاصة بها، تتألف من عنصر واحد فقط. وتتولى بنية بيانات "المجموعات المنفصلة" Disjoint Set إدارة هذه الدوائر باستخدام عمليتين أساسيتين:

- **Find**: يسأل مدينة: "من هو القائد المطلق (الأصل الجذري) لدائرتك الاجتماعية؟"

- **Union**: يجمع بين دائرتين اجتماعيتين منفصلتين تماماً تحت قيادة واحدة موحدة.

أثناء تكرار الخوارزمية على الحواف المرتبة، تأخذ معرفي مدينتين وتتحقق من جذورهما (كل منهما في دائرتها الاجتماعية أو مجموعتها الخاصة). إذا لم تكن جذورهما متساوية، فإنها تأخذ الحافة بينهما بأمان، ثم تدمج المجموعة ذات الترتيب الأصغر في المجموعة ذات الترتيب الأعلى، بحيث يكون جذر المجموعة ذات الترتيب الأعلى هو جذر المجموعة ذات الترتيب الأصغر. أما إذا كانت الجذور متساوية، فإنها تتخطى كليهما لأن لهما مساراً مشتركاً أقصر، وبالتالي لا يحتاجان إلى حافة مباشرة بينهما من شأنها أن تسبب حلقة مغلقة.

2. حساب Subgradient Vector (G): يمثل التدرج الفرعي الاتجاه الرياضي

لا كبر خطأ، لكل مدينة i ، نحسب كم تنحرف درجتها الحالية d_i في شجرة 1-Tree عن الهدف المراد في أي رحله TSP فعلية (حيث كل مدينة درجتها تساوي 2 تماماً). يمكننا تعريف الخطأ الهيكلي الفردي لأي عقدة i على النحو التالي:

$$G_i = d_i - 2$$

لفهم كيف يُمثل التدرج الفرعي الاتجاه الرياضي لأكبر خطأ، من المفيد الابتعاد عن البرمجة والنظر إلى هندسة التحسين. إذا كنت تتسلق جبلاً في ضباب كثيف وتريد الوصول إلى القمة بأسرع وقت ممكن، فإنك تنظر إلى الأرض تحت قدميك وتخطو في الاتجاه الذي يصعد فيه المنحدر بأكبر انحدار ممكن. في علم التفاضل والتكامل، يُسمى اتجاه الصعود الأكبر بالتدرج (Gradient - grad). ومع ذلك، فإن رسم دالة الهدف في تحسين Held-Karp ليس جبلاً أملساً ومستديراً تماماً. لأن الشجرة أحادية البعد تتكون من حواف حادة منفصلة، يبدو رسم الدالة وكأنه ماسة متعددة الأوجه أو مثل الهرم. يحتوي على حواف وزوايا حادة حيث يفشل حساب التفاضل والتكامل القياسي لأنه لا يمكنك حساب مشتقة تقليدية عند نقطة حادة. هنا يأتي دور التدرج الفرعي (Subgradient). حيث إنه تعميم للتدرج للدوال غير الملساء والمتعرجة.

في المرحلة الثانية، يتمثل هدفنا الأول هو جوله صالحه (Valid TSP Tour). في الجولة الصالحة، يكون لكل مدينة هدف بأن تكون درجتها تساوي 2 بالضبط. إذا أنتجت الخوارزمية 1-Tree حيث تكون درجة إحدى المدن 4، فإننا نواجه خطأً هيكلياً. أي أن المدينة مكتظة. إذا كانت درجة إحدى المدن تساوي 1، فإننا نواجه خطأً أيضاً؛ المدينة معزولة.

إذا كانت المدينة مزدحمة ($d_i = 4$)، يكون تدرجها موجباً ($4 - 2 = +2$). إذا كانت المدينة منعزلة أو طريق مغلق ($d_i = 1$)، يكون تدرجها سالباً ($1 - 2 = -1$). وإذا كانت المدينة في حاله دوره TSP مثاليه ($d_i = 2$)، ينهار تدرجها نحو الصفر ($2 - 2 = 0$).

لنتذكر صيغة إجمالي "التكلفة العقابية" المعممة الخاصة بنا:

$$W(\pi) = \sum_{(i,j) \in E'} C_{i,j} + \sum_{i=1}^N d_i \pi_i$$

لننظر كيف يؤثر تغيير عقوبة مدينة واحدة π_i على إجمالي التكلفة المعاقبة $W(\pi)$. إذا اعتبرنا حواف شجرة 1-Tree المختارة ثابتة مؤقتًا، فيمكننا حساب المشتقة الجزئية للتكلفة بالنسبة لعقوبة تلك المدينة:

$$\frac{\partial W}{\partial \pi_i} = d_i$$

يُشير هذا إلى أن معدل التغير الحالي (الميل) لدالة التكلفة على طول المحور π_i يساوي تماماً الدرجة الحالية للعقدة، d_i . ومع ذلك، فإننا نسعى لتعظيم الحد الأدنى، ولكن يجب علينا القيام بذلك دون السماح "للعقوبات" (penalties) بتشويه المخطط البياني إلى ما لا نهاية. وتتقضي مسألة التحسين المزدوج (dual optimization problem) تحقيق توازن في النظام، حيث تقوم خطوة التعديل المستهدفة بضبط الميل بالنسبة للقيد المفروض (أي الدرجة المستهدفة البالغة 2)؛ وعند طرح القيد المستهدف، يصبح متجه الميول الاتجاهية كما يلي:

$$\nabla W(\pi) \approx \begin{bmatrix} d_1 - 2 \\ d_2 - 2 \\ \vdots \\ d_N - 2 \end{bmatrix} = \begin{bmatrix} G_1 \\ G_2 \\ \vdots \\ G_N \end{bmatrix} = \vec{G}$$

هذا المتجه $\vec{G}$ هو التدرج الفرعي الخاص بنا.

يخبرنا Subgradient بالاتجاه الذي يجب سلكه لتعديل أخطائنا البنيوية وماهي النقاط التي علينا تعديلها. على سبيل المثال، إذا كانت المدينة

ذات درجة تساوي 5 ($G_i = +3$). يكون الميل ايجابى واتجاه subgradient vector يُؤشر بشده نحو الاتجاه الموجب. ومع التقدم في الحلقات, يسبب ذلك ازدياد قيمه π_i لتلك المدينه بشكل اقوى. وهذا يجعل الوصلات المتصله بتلك المدينه اكبر بالوزن واغلى مع كل حلقه, مما يجبر خوارزميه Kruskal على اقتطاع الوصلات عاليه التكلفه منها.

وعلى العكس تماماً, اذا كانت المدينه ذات درجه منخفضه مثل 1 ($G_i = -1$). في هذه الحاله يُؤشر متجه subgradient نحو الاتجاه السالب. ومع الوقت, ستخفض قيمه π_i لتلك المدينه. مما يجعل الوصلات المتصله معها اخص في التكلفه, ويجبر ذلك خوارزميه Kruskal على اضافته المزيد من الوصلات لتلك المدينه.

3. حساب حجم الخطوه (Step Size (t): لايمكننا ببساطه تغيير قيم π بشكل اعمى؛ فعل ذلك سيؤدي لانحراف النظام بشكل كبير وعشوائي عن القيمه الفضلى. يتم حساب حجم التغير عن طريق استخدام متغير setp-size scalar t, مقدار بواسطه Held and Karp's classic convergence formula :

$$t = \lambda \frac{\text{Target Bound} - W(\pi)}{\sum_{i=1}^N (G_i)^2}$$

حيث:

- λ Lmabda: هو معامل التخمين التكيفي والذي يبدأ حول عدد حقيقي ويتلاشى نحو الصفر.
- Target Bound: هو تقدير للقيم الحقيقه للسقف (Upper Bound), غالباً ما يحسب ابتداءً باستخدام خوارزميه بحث سريعه مثل nearest neighbor, او تعدل بواسطه الحلقه الخارجيه.
- المقام $\sum (G_i)^2$: مجموع الاخطاء للتربيع, ذلك كان الشكل العام لتوزيع المدن عشوائياً والدرجات بعيدة عن 2, تكبر قيمه المقام, مما يجعل حجم الخطوه اصغر واكثر استقراراً.

4. تحديث قيم π : اخيراً, يتم تعديل مصفوفه قيم العقوبات (او يتم تعديلها لكل مدينه) من اجل دوره التاليه:
اذا كانت درجه المدينه عاليه, قيمه π خاصتها تزداد. في دوره التاليه, كل

الوصلات المتصلة معها تصبح "أعلى"، يدفع ذلك خوارزميه Kuraskal على تجنبها. وبالعكس، المدن المنفيه او ذات الوصله الواحده يحصلون على قيمه تعديل (π) سالبه، مما يجعل وصلاتهم "ارخص" واكثر جذباً للخوارزميه في دوره التاليه.

4.2 الخطوة الثالثه: The Alpha (α) Transformation and Candidate Set Generation

في هذه المرحله يحدث السحر. حيث تستخدم شجرة 1-Tree المُحسَّنة كمرشح رياضي لحذف ما يقرب من 95% إلى 99% من الحواف في الرسم البياني بشكل دائم، مما يحول المشكله من مشكله مستحيله حسابياً إلى سلسله من عمليات البحث المحليه فائقة السرعة.

مفهوم تقارب ألفا (alpha-nearness) في تحليل حساسية الرسم البياني (Graph Sensitivity Analysis) في التحسين التوافقي (Combinatorial Optimization)، يتطلب تحديد الوصلات ذات الاحتمالية الأعلى للانتماء إلى مسار البائع المتجول الأمثل يتجاوز المسافه المكانية المجردة. فبينما توفر الأوزان المُعاقَبة C'_{ij} الناتجة عن عمليه Held-Karp Relaxation حدًا أدنى ممتازًا (Best Lower Bound)، إلا أنها لا تقيس بشكل صريح القيمة الفردية للوصلات غير الشجرية. ولحل هذه المشكله، تعتمد خوارزميه Lin-Kernighan-Helsgaun (LKH) على مقياس "حساسيه" يُعرف باسم تقارب ألفا alpha-nearness (أو قيمه الندم).

1.4.2 ما هي قيم ألفا (α)؟

في نظريه Graph Thoery التقليديه، إذا أردنا معرفه ما إذا كانت احدى الوصلات جيده، ننظر إلى مقدار المسافه خاصتها. أما في خوارزميه LKH-3، ننظر إلى قيمه ألفا (المعروفه أيضاً بالتكلفه الزائده أو قيمه الندم).

تُعرَّف قيمه ألفا للوصله (i, j) رياضياً كما يلي: ما مقدار الزيادة في التكلفه الإجماليه لشجرة الحد الأدنى 1-Tree إذا أُجبرنا الخوارزميه على تضمين (اضافه) هذه الحافه تحديداً؟

2.4.2 التعريف الرياضي للندم (Regret)

لنفترض أن $W(\pi)$ يمثل إجمالي التكلفة العقابيه (Penalized Cost) لشجرة 1-Tree المثلى التي تم اكتشافها في نهاية مرحلة تحسين التدرج الفرعي (Subgradient Optimaization). لنفترض أننا نختار وصله عشوائية (i, j) من المجموعة العامة لجميع الوصلات الممكنة E ، ونفرض قيّدًا صارمًا: يجب على الخوارزمية توليد شجرة أحادية 1-Tree جديدة تتضمن الوصله (i, j) . ولنرمز إلى تكلفة هذه الشجرة الأحادية المقيدة (Constrained 1-Tree) حديثًا بـ $W_{i,j}(\pi)$. تُعرّف قيمة تقارب ألفا للوصله (i, j) على أنها الفرق بين هاتين الحالتين:

$$\alpha(i, j) = W_{i,j}(\pi) - W(\pi)$$

بما أن $W(\pi)$ تمثل الحد الأدنى العام غير المقيد، فإن الشجرة المقيدة $W_{i,j}(\pi)$ يجب أن تكون أكبر من أو تساوي $W(\pi)$ تمامًا. لذلك، فإن $\alpha(i, j)$ دائمًا قيمة غير سالبة ($\alpha \geq 0$).

3.4.2 التفسير الطوبولوجي لقيم ألفا: Topological Interpretation

يوفر مقدار قيمة ألفا للوصله تقييماً نظرياً دقيقاً لمدى فائدتها:

- $\alpha(i, j) = 0$: تُعدّ الحافة (i, j) عنصراً طبيعياً موجوداً في شجرة 1-Tree المثلى. ولا يُغيّر إدراجها (الوصله) قسراً شيئاً، ما يؤدي إلى انعدام التكلفة الزائدة. وتمثل هذه الوصلات العمود الفقري للشبكة المثلى.
- قيمه $\alpha(i, j)$ صغيره: لا يقع هذا الخط في شجرة 1-Tree الحالية، ولكن إدخالها بالقوه في البنية لا يُحدث سوى زيادة طفيفة في الحد الأدنى العام. تتميز هذه الخطوط بتنافسية عالية من الناحية الرياضية، ولها احتمالية كبيرة للظهور في المسار الأمثل الحقيقي.
- قيمه $\alpha(i, j)$ كبيره: يؤدي فرض وجود حافة إلى تعطيل بنية الشجرة 1-Tree بشكل جذري، مما يتسبب في ارتفاع التكلفة الإجمالية بشكل كبير. حتى لو كانت الوصله قصيرة جغرافياً، فإن قيمة ألفا العالية تشير إلى أنها طريق مسدود طوبولوجياً أو فخ هيكلي.

باستخدام هذا المقياس، ينتقل LKH من قياس مطلق للمسافة إلى قياس نسبي للضرورة الهيكلية.

4.4.2 الآلية الخوارزمية لحساب قيم ألفا: The Cycle-Exchange Theorem

على الرغم من أن التعريف النظري لتقارب ألفا - أي إدخال حافة في الشجرة 1-Tree وقياس فرق التكلفة الإجمالي - سليم من الناحية المفاهيمية، إلا أن حسابه بطريقة مباشرة غير عملي من الناحية الحسابية. يتطلب استخدام أسلوب البحث الشامل تشغيل خوارزمية Kruskal $O(n^2)$ مرة (مرة واحدة لكل وصله محتملة)، مما ينتج عنه تعقيد زمني إجمالي هائل.

لحل هذه المشكلة، تعتمد خوارزمية (LKH) Lin-Kernighan-Helsgaun على خاصية أساسية في طوبولوجيا الرسم البياني تُعرف بالدورة الأساسية The Fundamental Cycle. تسمح هذه الخاصية للخوارزمية بحساب قيمة ألفا لأي وصله في زمن ثابت $O(1)$ ، بشرط تنفيذ خطوة حسابية مسبقة واحدة عالية الكفاءة سيتم شرحها لاحقاً.

• المبدأ الطوبولوجي لإنشاء الدورة: تحتوي الشجرة الممتدة الدنيا MST المكونة من N عقدة على $N - 1$ وصله بالضبط، ولا تحتوي على حلقات مغلقة. وبحسب تعريف طوبولوجيا للشجرة، يوجد مسار واحد متصل وفريد يربط أي عقدتين عشوائيتين، i و j . ولنرمز لهذا المسار الفريد بالرمز $P(i, j)$. وبحسب التعريف، تسبب إضافة أي وصله جديدة (i, j) إلى شجرة موجودة دورة مغلقة واحدة فقط. لاستعادة بنية الشجرة الأصلية (وإزالة الدورة المغلقة) مع الحفاظ على أقل تكلفة ممكنة، يجب على الخوارزمية إزالة الحافة الأعلى تكلفةً الموجودة على مسار الشجرة الأصلي الذي يربط العقدة i بالعقدة j .

• تكلفة ترميم أو اصلاح الشجره (The Exchange Move): لإعادة الشجره إلى البنية الصحيحة، يجب حذف حافة واحدة فقط من هذه الدورة المُشكلة حديثاً. ولأن هدف الخوارزمية هو إيجاد أقل تكلفة ممكنة لهذه الشجرة المُقيدة حديثاً، يجب عليها استبعاد الحافة الأكثر تكلفة حسابياً ضمن تلك الدورة. وبما أن الوصله الجديدة (i, j) مُجبرة على البقاء، يجب اختيار الوصله المحذوفة من مسار الشجرة الأصلي $P(i, j)$.

لذا، فإن التغير الصافي في التكلفة الإجمالية للشجرة (وهو تعريف α تحديداً) هو ببساطة تكلفة الحافة التي أضفناها، مطروحاً منها تكلفة الحافة التي حذفناها. وهذا يُعطينا صيغة الحساب المباشر:

$$\alpha(i, j) = C'_{ij} - \max_{(u,v) \in P(i,j)} C'_{uv}$$

حيث:

- C'_{ij} يمثل الوزن العقابي (Penalized Cost π_i) للوصلة الجديدة التي يتم إدخالها بالقوة في الشجرة.

- $\max_{(u,v) \in P(i,j)} C'_{uv}$ يمثل وزن الوصلة الأكبر (الأثقل) الموجودة حالياً على المسار $P(i, j)$ داخل الشجرة.

• استراتيجية الحساب المسبق ($O(N^2)$): بينما تُحصر الصيغة أعلاه الحساب في مسار واحد، فإن فحص كامل الشجرة للعثور على الوصلة الأكبر (الأثقل) لكل زوج (i, j) على حدة سيظل مكلفاً حسابياً.

بدلاً من البحث عن المسار عند الحاجة، تحسب خوارزمية LKH مسبقاً هذه الحواف القصوى للرسم البياني بأكمله في آنٍ واحد باستخدام خوارزمية بحث مُحسَّنة (عادةً ما تكون خوارزمية البحث بالعرض Breadth-First Search أولاً أو خوارزمية البحث بالعمق أولاً Depth-First Search).

تُنَفَّذ العملية كما يلي:

تختار الخوارزمية العقدة i لتكون "جذر" الشجرة Tree Root. تبدأ الخوارزمية عملية بحث للخارج على طول فروع الشجرة. عند زيارة العقدة المجاورة v عبر الوصلة (u, v) ، فإنها تتعقب باستمرار أثقل وصله تمت رؤيتها حتى الآن. أقصى وزن للمسار إلى العقدة الجديدة v هو ببساطة أكبر قيمتين: أعلى وزن مسجل حتى العقدة السابقة u ، أو وزن الحافة المتصلة مباشرة (u, v) . تسجل الخوارزمية هذه القيمة القصوى في جدول بحث ثابت بحجم $N \times N$. رياضياً، تكون علاقة التكرار أثناء الاجتياز كما يلي:

$$MaxPath(i, v) = \max(MaxPath(i, u), C'_{uv})$$

من خلال تحديد جذر مسار عند كل عقدة من العقد N ، تُنشئ الخوارزمية مصفوفة كاملة $N \times N$ من أقصى المسارات في زمن $O(N^2)$ بالضبط. بمجرد إنشاء هذه المصفوفة في الذاكرة، يُصبح حساب قيمة التقارب α لأي حافة نظرية (i, j) إلى عملية طرح حسابية بسيطة وفورية.

هذا التحويل من إعادة بناء الرسم البياني في كل مره واستخراج القيم $(O(N^4))$ إلى بحث برمجي ديناميكي ذكي $(O(N^2))$ هو الآلية المعمارية المحددة التي تسمح لخوارزمية LKH-3 بتصفية الشبكات الضخمة إلى مجموعات فرعية من 5 مرشحين دون إبطاء المعالج.

5.4.2 تبسيط الرسم البياني وبناء مجموعات المرشحين

بينما توفر مصفوفة ألفا درجة حساسية دقيقة رياضياً لكل اتصال محتمل في الشبكة، فإن استخدام هذه المصفوفة $N \times N$ مباشرةً خلال مرحلة التوجيه يبقى غير عملي حسابياً. يهدف الجزء الثالث إلى تحويل قيمة الندم العددي المستمر (تقارب ألفا) إلى بنية طوبولوجية منفصلة ومتفرقة للغاية تُعرف باسم الرسم البياني للمرشحين Candidate Graph.

تعزل هذه العملية بشكل منهجي الحواف الأكثر أهمية مع استبعاد الاتصالات غير ذات الصلة إحصائياً بشكل دائم، مما يحد من نطاق البحث القادم من $O(N^2)$ إلى $O(N)$.

• التعريف الرياضي لمجموعة المرشحين: لأي عقدة معينة i في رسم بياني كامل $G = (V, E)$ ، شمل d الجوار الكامل لها $N - 1$ وصله ممكنة. تُعرّف مجموعة المرشحين $C(i)$ على أنها مجموعة فرعية محدودة تمامًا من هذه العقد المتجاورة، تحتوي فقط على k من أكثر الوصلات الواعدة وفقاً لمصفوفة ألفا.

لإنشاء مجموعته $C(i)$ ، تُقيّم الخوارزمية مجموعة جميع العقد المتجاورة $j \in V \setminus \{i\}$ وترتيبها ترتيباً تصاعدياً بناءً على معايير تقييم متعددة المستويات:

1. المقياس الأساسي (تقارب ألفا): يتم تصنيف العقد بشكل أساسي بناءً على قيم ألفا $\alpha(i, j)$. يتم وضع الوصلات حيث $\alpha(i, j) = 0$ (والتي تنتمي أصلاً إلى شجرة Tree-1 المثلّي) في أعلى التسلسل الهرمي.
2. كسر التعادل من الدرجة الثانية (الوزن العقابي): إذا كانت وصلات متعددة تشترك في قيمة ألفا أي لها نفس القيمة (وهو أمر شائع للغاية بالنسبة لـ ألفا = 0)، فسيتم فرزها فرعياً حسب وزنها العقابي وفقاً لـ C'_{ij} , Held-Karp.
3. كسر التعادل من الدرجة الثالثة (المسافة الهندسية): في حالة نادرة من التعادل في الوزن العقابي، تعمل المسافة المكانية الخام C_{ij} ككسر للتعادل النهائي.

تقوم الخوارزمية بتقصير هذه القائمة المرتبة، مع الاحتفاظ فقط بعدد وصلات اقصى (مثل اقرب 5 مرشحين) في اعلى القائمة وتجاهل الباقي.

• The Theoretical Bound: اختيار حد المرشحين يمثل حجم مجموعة المرشحين، $|C(i)|$ ، مفاضلة حاسمة بين الحفاظ على الحل الأمثل وسرعة التنفيذ. إذا كان حجم $|C(i)|$ كبيراً جداً، فإن عامل التفرع لمحرك البحث المحلي اللاحق سيؤدي إلى تضخم هائل في عدد الاحتمالات. أما إذا كان حجم $|C(i)|$ صغيراً جداً، فإن الخوارزمية تُخاطر بحذف وصله تنتمي فعلياً إلى مسار البائع المتجول الأمثل، مما يحرم الحل رياضياً من الوصول إلى الحد الأدنى غير المحلي (اي global minimum).

من خلال تحليل تجريبي مُعمّق على مجموعات البيانات في مكتبه TSPLIB، أثبت Helsgaun أن مقياس π -nearness Held-Karp دقيق للغاية لدرجة أن الوصلات المثلى الحقيقية تُصنّف دائماً تقريباً ضمن أفضل 5 مراكز. ونتيجةً لذلك، يُفرض المعامل القياسي في LKH-3 حدًا صارمًا قدره:

$$|C(i)| \leq 5$$

من خلال تقييد كل عقدة بعدد مرشحين اقصى يبلغ 5 مرشحين داخل الرسم البياني، يتقلص فضاء البحث إلى حد أعلى يبلغ 5^N وصله، مما يجعل التعقيد الزمني لعمليات البحث المحلية خطياً تماماً بالنسبة لحجم الرسم البياني.

5.2 الخطوة الرابعة: محرك البحث المحلي المتغير k-opt

تعتمد طرق البحث المحلية التقليدية لحل مسألة البائع المتجول TSP، مثل opt-2 و opt-3، على نطاقات طوبولوجية محددة مسبقًا. يُقيّم البحث المحدود مسبقًا جميع التوليفات الممكنة لتبديل k من الحواف. ومع ازدياد قيمة k ، يتزايد التعقيد الحسابي أُسّيًا وفقًا لـ $O(N^k)$ ، مما يجعل حساب الأعماق العشوائية مستحيلًا في الشبكات الكثيفة. ونتيجةً لذلك، غالبًا ما تقترب الطرق الثابتة قبل الأوان، فتقع في الحد الأدنى المحلي (Local Minima) حيث لا يُحقق أي تبديل k-opt صالح تخفيضًا هندسيًا إيجابيًا في اجمالي المسافة، حتى لو تمكن تبديل من رتبة أعلى (مثل opt-8 أو opt-12) من تجاوز هذا المأزق الطوبولوجي.

يحل أسلوب Lin-Kernighan (LK) هذه المشكلة الهيكلية جذريًا باستبدال تقييم النطاق الثابت بتبديل تسلسلي ذي عمق متغير. بدلًا من تحديد تعقيد عملية التبديل مسبقًا، تقوم الخوارزمية بإنشاء تحركات opt-k المعقدة بشكل ديناميكي، مما يسمح لعمق البحث بالتكيف في الوقت الفعلي بناءً على الوعد الرياضي للبحث.

مع تحديد مساحة البحث رياضيًا بواسطة قوائم المرشحين غير الموجهة، تنتقل خوارزميه Lin-Kernighan-Helsgaun (LKH) إلى التوجيه الفعلي. تبدأ الخوارزمية بإنشاء مسار أولي صالح، قد لا يكون مثاليًا (عادةً عبر بحث جشع مثل Depth-First Search عبر وصلات المرشحين). ثم يتحول الهدف إلى تحسين هذا المسار بشكل متكرر باستخدام طريقة البحث المحلي متغير العمق Lin-Kernighan variable-depth local search heuristic.

1.5.2 الأساس الرياضي لتبديلات k-opt

تعمل خوارزميات البحث المحلي على تحسين مشكله البائع المتجول من خلال تنفيذ اضطرابات طوبولوجية مستمرة، تُعرف باسم تبديلات الوصلات.

لنفترض أن T تمثل المسار الصحيح الحالي. تتضمن حركة $k-opt$ القياسية تحديد مجموعة من k وصله بالضبط في المسار الحالي، $X = \{x_1, x_2, \dots, x_n\}$ ، وقطعها. يؤدي هذا الإجراء إلى تقسيم المسار المتصل إلى k مسار منفصل. لاستعادة صحة المسار، تُدخل الخوارزمية مجموعة جديدة من k حافة، $Y =$

$\{y_1, y_2, \dots, y_n\}$ ، وتعيد توصيل نقاط النهاية المكشوفة في مسار مغلق جديد، T' . يتم تنفيذ عملية التبديل إذا وفقط إذا كانت التكلفة المكانية الإجمالية للحواف المُدخلة حديثاً أقل تماماً من تكلفة الحواف المقطوعة، مما ينتج عنه انخفاض هندسي إيجابي (أو "مكسب") في طول المسار الإجمالي:

$$\Delta = \sum_{i=1}^k C(x_i) - \sum_{i=1}^k C(y_i) > 0$$

حيث تشير $C(e)$ إلى تكلفة المسافة للحافة e . إذا كانت $\Delta > 0$ ، يتم قبول الحركة، ويتم استبدال T بـ T' ، وتبدأ دورة التحسين من جديد.

2.5.2 قيود خوارزميات k-opt الثابتة

تعتمد طرق البحث المحلية التقليدية، مثل خوارزمية 2-opt أو 3-opt، على عمق ثابت ومحدد مسبقاً. إذ لا تُقيّم خوارزمية 2-opt سوى أزواج الوصلات، بينما تُقيّم خوارزمية 3-opt ثلاثيات الوصلات.

يكنم العيب الجوهرى في تحسين العمق الثابت Fixed-Depth Optimization في ظاهرة الحد الأدنى المحلي Local Minima. فقد تصل عملية البحث إلى حالة لا يُمكن فيها لأي توزيعه ممكنة من تبديل حافتين أو ثلاث أن تُنتج قيمة Δ موجبة. عندئذٍ، تتوقف الخوارزمية، مُفترضةً أنها وجدت الحل الأمثل. مع ذلك، قد تمتلك عملية تبديل معقدة للغاية، مثل خوارزمية 6-opt أو 8-opt، القدرة على كسر الجمود وتحقيق مسافة أقصر. ويُعدّ برمجة بحث 8-opt ثابتاً أمراً مستحيلاً حسابياً، نظراً لأن عامل التفرع لتبديلات 8 حواف عبر رسم بياني كبير مرتفع للغاية.

3.5.2 ابتكار Lin-Kernighan: التبديل التسلسلي ذو العمق المتغير

تتجاوز خوارزمية Lin-Kernighan (LK) مشكلة اعماق البحث الثابتة من خلال تقديم تبديل تسلسلي k-opt متغير. فبدلاً من تقييم جميع الحواف k في آن واحد، تُنشئ LK التبادل ديناميكياً، وصلةً تلو الأخرى، مُشكّلةً تسلسلاً متناوباً

من الاتصالات المقطوعة والمضافة.

تبني الخوارزمية مسارًا متناوبًا من الحواف:

$$x_1, y_1, x_2, y_2, \dots, x_i, y_i$$

عند كل مستوى عمق i : تقطع الخوارزمية وصله مسار موجودة x_i . وتضيف حافة مرشحة جديدة y_i . وتقيم ما إذا كان بالإمكان إغلاق التسلسل في مسار صالح عن طريق محاكاة إضافة وصله إغلاق نهائية، y_i^* . إذا حقق إغلاق المسار عند العمق الحالي i مكسبًا إيجابيًا، يُسجل المسار الجديد كأفضل حل معروف. مع ذلك، لا تتوقف الخوارزمية. بل تستمر في التعمق أكثر في شجرة البحث، مستكشفةً $i+1$ و $i+2$ وما بعدها، ومقيمةً ديناميكيًا تحركات k -opt ذات تعقيد عشوائي (غالبًا ما تصل إلى أعماق $k = 10$ أو أكثر) لمعرفة ما إذا كان بالإمكان اكتشاف مكسب أكبر في أسفل الفرع التسلسلي.

تُنشئ عملية التبادل المتسلسل حركة k -opt حافةً تلو الأخرى، مُشكّلةً تسلسلاً متناوبًا من الروابط المقطوعة والروابط المضافة. لنفترض أن المسار الصحيح الحالي هو T . تبدأ الخوارزمية باضطراب طوبولوجي عن طريق اختيار عقدة أساسية t_1 وقطع حافة مسار موجودة $x_1 = (t_1, t_2)$. يؤدي هذا إلى تقسيم المسار إلى مسار هاميلتوني مفتوح. لبدء إصلاح البنية، تختار الخوارزمية حافة مرشحة جديدة $y_1 = (t_2, t_3)$ لإضافتها إلى التسلسل.

تعمّم هذه العملية بشكل تكراري. عند أي مستوى عمق مُعطى i ، تُوسّع الخوارزمية المسار المتناوب عن طريق: قطع حافة مسار أصلية $x_i = (t_{2i-1}, t_{2i})$. إضافة حافة مرشحة جديدة $y_i = (t_{2i}, t_{2i+1})$.

ينتج عن ذلك سلسلة متتابعة من الحواف المتناوبة: $x_1 y_1 x_2 y_2 \dots x_i y_i$. ولأن الحواف تُختار بالتتابع بدلاً من اختيارها في وقت واحد، يتم تجاوز الانفجار التوافقي المرتبط بتقييم k من الحواف المستقلة في وقت واحد بشكل كامل.

4.5.2 معيار الربح الإيجابي (تحديد حدود شجرة البحث)

لمنع هذه المتتالية التكرارية من البحث بلا نهاية، قدّم لين وكيرنيغان قاعدة تقليم رياضية صارمة تُعرف بمعيار الربح الموجب. لنفترض أن g_i هو الربح

الفردى لخطوة واحدة فى المتتالية:

$$g_i = C(x_i) - C(y_i)$$

إن الربح التراكمى G_i عند أى عمق i هو مجموع مكاسب الخطوات الفردية:

$$G_i = \sum_{j=1}^i (C(x_j) - C(y_j))$$

يُسمح للخوارزمية بمواصلة تعميق البحث إلى $i + 1$ فقط إذا وفقط إذا ظل المكسب التراكمى موجباً تماماً ($G_i > 0$). يضمن هذا القيد أن يظل المسار المفتوح الوسيط تنافسياً هندسياً. عندما يصبح $G_i \leq 0$ ، يُثبت رياضياً أن التبادل التسلسلى طريق مسدود، ويتم حذف الفرع التكرارى فوراً.

5.5.2 الاتحاد مع مجموعه المرشحين

تتجلى الكفاءة الحسابية الحقيقية لمحرك خوارزميه LKH-3 فى كيفية تفاعل المرحلة الرابعة مع المرحلة الثالثة. عندما يحاول التبديل التسلسلى اختيار حافة جديدة y_i لإضافتها إلى المسار المتناوب، فإنه لا يقوم بمسح الرسم البيانى العام. يقتصر اختيار y_i بشكل صارم على الوصلات الموجودة فى مجموعة المرشحين للعقدة الحالية. من خلال إجبار التبديل التسلسلى على تقييم أفضل 5 حواف فقط وفقاً لمصفوفة Held-Karp α -nearness، يتم كبح عامل التفرع التكرارى بشكل كبير. بذلك يستطيع محرك Len-Kernighan الغوص ديناميكياً إلى أعماق k-opt هائلة فى أجزاء من الثانية لأنه يستكشف فقط التركيبات الطوبولوجية التى اجتازت رياضياً عملية ترشيح قيم الندم عند الحد الأدنى (Lower Bound).

6.5.2 اليه تبديل المسارات

تُنشئ عملية التبديل المتسلسل حركة k-opt وصلةً تلو الأخرى، مُشكّلةً تسلسلاً متناوباً من الروابط المقطوعة والروابط المضافة. لنفترض أن المسار

الصحيح الحالي هو T . تبدأ الخوارزمية باضطراب طوبولوجي عن طريق اختيار عقدة أساسية t_1 وقطع حافة مسار موجودة $x_1 = (t_1, t_2)$. يؤدي هذا إلى تقسيم المسار إلى مسار هاميلتوني مفتوح (Open Hamiltonian Path). لبدء إصلاح البنية، تختار الخوارزمية حافة مرشحة جديدة $y_1 = (t_2, t_3)$ لإضافتها إلى التسلسل. تعمّم هذه العملية بشكل تكراري. عند أي مستوى عمق مُعطى i ، تُوسّع الخوارزمية المسار المتناوب عن طريق:

$$\begin{aligned} \text{قطع حافة مسار أصلية } x_i &= (t_{2i-1}, t_{2i}) \\ \text{إضافة حافة مرشحة جديدة } y_i &= (t_{2i}, t_{2i+1}) \end{aligned}$$

ينتج عن ذلك سلسلة متتابة من الحواف المتناوبة: $x_1 y_1 x_2 y_2 \dots x_i y_i$. ولأن الحواف تُختار بالتتابع بدلاً من اختيارها في وقت واحد، يتم تجاوز الانفجار التوافقي المرتبط بتقييم k من الحواف المستقلة في وقت واحد بشكل كامل.

6.2 Backtracking and the LKH-3 Extension

بينما اقتصرت خوارزمية LK الأصلية على التحركات المتسلسلة في مسارات متناوبة بسيطة، يُحسّن إطار عمل LKH الخاص بكيلد هيلسغاون هذه البنية من خلال دمج آليات تراجع Backtracking واسعة النطاق.

إذا لم يُسفر تسلسل ما عن مسار مُحسّن وتجاوز حد الربح الإيجابي، فإن خوارزمية LKH-3 تتراجع بشكل منهجي لأعلى شجرة البحث، مُستبدلةً حوافاً مرشحة بديلة (مثل استبدال y_i بحافة محلية مرشحة مختلفة) لاستكشاف الفروع المتوازية. علاوة على ذلك، ولأن اختيار y_i يخضع حصرياً لمجموعات المرشحين ذات التقارب ألفا المتفرقة من المرحلة الثالثة، فإن محرك العمق المتغير هذا قادر على تنفيذ عمليات تبادل متسلسلة تتجاوز عمق $k = 50$ بسهولة. يمنح هذا التوليف بين ترشيح ألفا والتكرار ذي العمق المتغير الخوارزمية قدرةً لا مثيل لها على تجاوز الحد الأدنى المحلي العميق وعزل الحل الأمثل العالمي الحقيقي.